Data Mining & Machine Learning
M.Sc. in Artificial Intelligence and Data Engineering
University of Pisa

Project Report

Academic Year 2025–2026

Project Title: FakeNews Detector

Authors
Alessio Grillo — Student ID: 654947
Andrea Giacomazzi — Student ID: 720164
Submission date: 12/06/2026

Abstract

This study proposes an automated binary classifier to combat digital misinformation using the WELFake dataset (72,095 articles). We developed a dual-feature pipeline that combines semantic TF-IDF representations with structural stylometric features, applying strict zero-knowledge MinMax scaling to prevent data leakage. Among the evaluated algorithms, an optimized LightGBM model achieved the highest performance, with both accuracy and F1-score reaching 97.97% on the unseen test set. To ensure decision transparency, we integrated a hybrid Explainable AI (XAI) framework, employing SHAP for global model validation and LIME for local, real-time explanations. Ultimately, this system delivers a highly accurate and interpretable solution for fake news detection.

1. Introduction

The digital era has driven an explosion of data, but also the rapid spread of misinformation. Social media and digital news platforms heavily accelerate the reach and impact of these "fake news" stories [1]. This deception carries profound consequences: it manipulates public opinion, polarizes communities, and undermines trust in democratic institutions. Because the sheer volume of daily information makes manual fact-checking impossible, developing automated detection systems using Natural Language Processing (NLP) and Machine Learning is now a scientific priority [2]. To address this challenge, we introduce *FakeNews Detector*, an automated system built to identify and flag non-credible articles. By analyzing both the semantic content and structural patterns of the text, our architecture captures the nuances of digital deception, delivering high predictive accuracy alongside human-readable transparency.

This project aims to:

1. Develop a robust predictive pipeline for fake news detection that fuses semantic text representations (TF-IDF) with normalized stylometric features.
2. Conduct a comparative assessment of multiple machine learning algorithms and perform hyperparameter tuning on selected models to identify the best-performing classifier for the binary classification task.
3. Retrain the best-performing model on the entire training set and evaluate its final performance on the unseen test set.
4. Provide global and local model explanations using Explainable AI (XAI) frameworks, specifically SHAP and LIME, to guarantee the transparency of predictions and foster user trust.

2. Problem

This project detects misinformation in digital media using a machine learning framework. We define the task as supervised binary classification. The proposed pipeline processes raw news articles and predicts their veracity.

Table 1. Problem and dataset specifications.

Attribute	Specification
Task type	Supervised binary classification
Dataset	WELFake
Total samples	72,134 independent records
Raw input features	Index (ID), Title (String), Text Body (String)
Feature types	Unstructured textual data
Target variable	Article veracity (0: Fake News, 1: Real News)
Missing values	39 missing text; 558 blank titles retained
Class distribution	Balanced (Class 0: 51.5%, Class 1: 48.5%)
Evaluation metrics	Accuracy, F1-score, Precision, Recall, AUC-ROC
Data split	80% Training, 20% Testing (Stratified sampling)

2.1 Exploratory Data Analysis

The WELFake dataset is well-balanced, comprising 51.5% fake and 48.5% real news. Missing data is negligible: only 0.05% (39 samples) lack body text and 0.77% (558 samples) lack titles. Notably, all missing values occur exclusively within the real news class, where 0.11% of texts and 1.5% of titles are absent. Figure 1 highlights key stylometric differences between the classes. The *title_uppercase_ratio* provides the clearest class separation, showing a bimodal concentration for fake news (red) and a broader distribution for real news (blue). In contrast, *text_length* and *spelling_errors_ratio* display heavily overlapping, right-skewed profiles. Both exclamation ratios are intensely zero-inflated, with fake news strictly clustered at zero and real news showing a slightly longer tail. Figure 2 summarizes the primary linguistic patterns based on n-gram frequencies. Although both classes frequently reference political entities, fake news heavily overuses procedural tokens (e.g., over 175,000 occurrences of “said”) and often retains metadata artifacts (e.g., “featured image”, “twitter com”). Conversely, real news is easily distinguished by official press agency signatures (e.g., “washington reuters”).

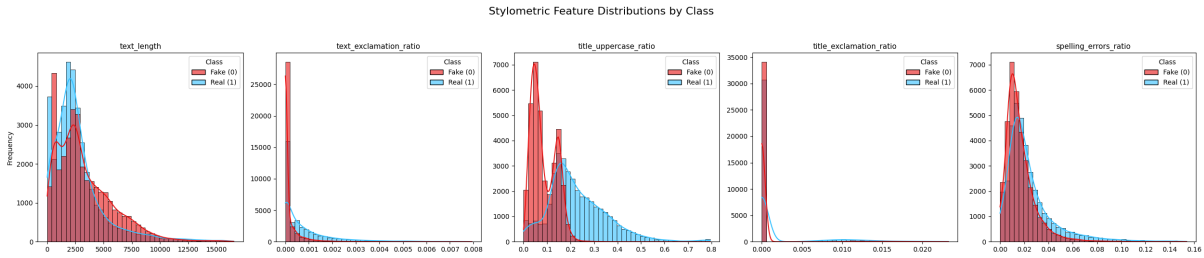

Figure 1. Stylometric feature distributions by class

2.2 Feature Description

Our raw data originates from the public WELFake dataset [3], a comprehensive corpus of 72,134 samples. Each record initially provides the following raw fields:

- **Serial Number (Numerical):** A unique identifier used strictly for indexing and dropped prior to training.
- **Title (Textual):** The article’s headline. Deceptive headlines often rely on clickbait structures or exaggerated capitalization to capture attention.
- **Text (Textual):** The main body content, capturing the core narrative where fake news typically exhibits partisan buzzwords compared to objective reporting.

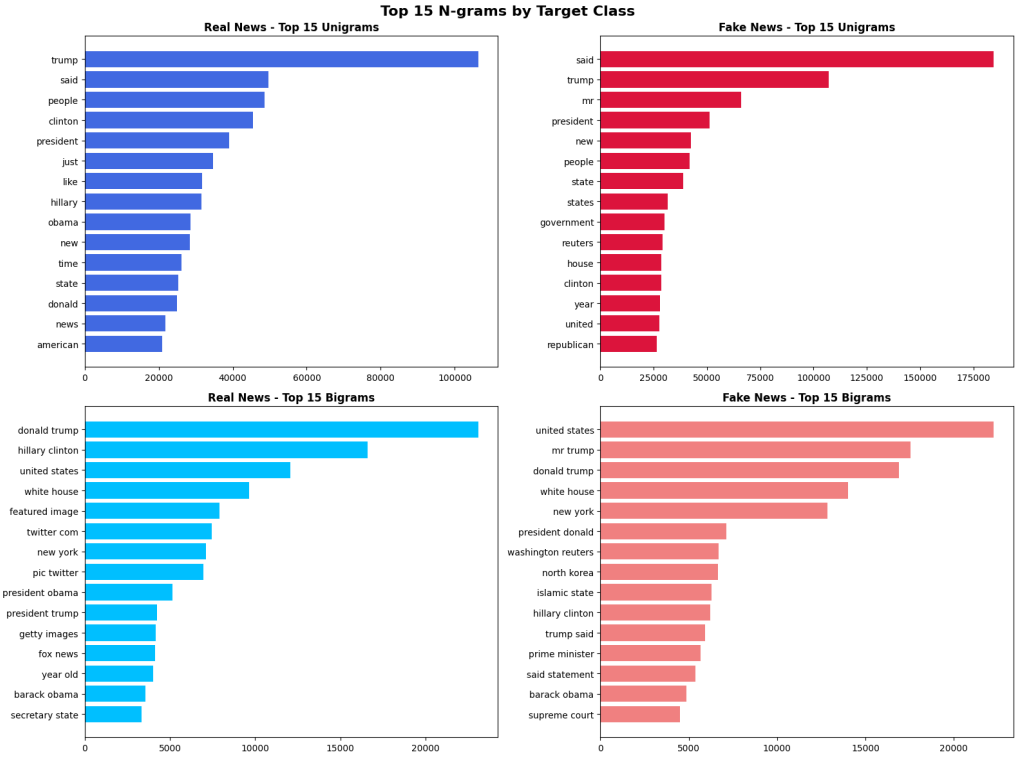

Figure 2. Top unigram and bigram frequencies highlighting linguistic differences and data artifacts across classes.

The target variable is a binary label:

- **Label (Binary):** The verified veracity of the document (1 for real news, 0 for fake news).

To prepare the raw text for our machine learning models, we transformed the *Title* and *Text* fields into 10,005 numerical features (10,000 TF-IDF terms and 5 stylometric metrics), as detailed in Section 3.2 Table 1 summarizes the problem and dataset characteristics.

3. Pipeline and Experiments

3.1 Preprocessing

We removed 39 records missing the main text body to avoid introducing artificial patterns via imputation, preserving the overall class balance. Conversely, we retained 558 records with missing titles to preserve their high-quality body content and maintain the size of our training corpus. We then concatenated the title and text into a single unified string, enabling the model to process headlines and article bodies simultaneously. Next, we normalized the unified text using the NLTK library through the following sequential steps:

- **Noise Removal:** Stripped non-alphanumeric characters and punctuation via regular expressions.
- **Case Folding:** Converted all text to lowercase.
- **Tokenization:** Segmented the text strings into discrete word tokens.
- **Lemmatization:** Reduced tokens to their morphological roots, shrinking the feature space dimensionality while retaining semantic meaning.

- **Stopword Elimination:** Removed standard English stopwords alongside domain-specific artifacts (e.g., “said”, “twitter com”, “featured image”) identified in the n-gram frequency graph to prevent contextual data leakage.

We encoded the normalized text into a numerical matrix using TF-IDF vectorization. Additionally, we applied a MinMax Scaler to the five engineered stylometric features. Bounding these variables between 0 and 1 aligns their magnitude with the sparse TF-IDF matrix, preventing scale-sensitive algorithms from misinterpreting feature importance. Finally, to prevent data leakage, we enforced a strict pipeline order by splitting the dataset into training and testing subsets *before* applying vectorization and scaling. We fitted the TF-IDF vectorizer and MinMax scaler exclusively on the training subset, and then used those learned parameters to transform the test set. This guarantees that test set distributions do not influence the feature engineering process.

3.2 Feature Selection and Engineering

Our final feature space comprises 10,005 numerical variables, combining semantic and stylometric features to create a comprehensive mathematical representation of the documents without introducing redundant dimensions.

- **Semantic Features (10,000 variables):** We applied TF-IDF vectorization (unigrams and bigrams) to the concatenated text (combining the Title and Text fields). By restricting the vocabulary to the top 10,000 most frequent tokens, we effectively filtered out uninformative lexical noise while isolating the core narrative and partisan buzzwords.
- **Stylometric Features (5 variables):** To capture the rigid structural anatomy of deceptive media, we engineered five features from the raw text: title uppercase ratio (clickbait capitalization), text character length (article size), spelling error rate (editorial quality), and exclamation/question mark densities (sensationalist tone). We applied MinMax scaling to align their numerical magnitudes with the sparse TF-IDF matrix.

Figure 3 illustrates the top 20 features ranked by their Chi-Square scores. This ranking validates our combined feature engineering approach, demonstrating that both structural anomalies (e.g., *title_uppercase_ratio*) and high-impact political vocabulary (e.g., “reuters”, “hillary”) are highly informative for distinguishing between real and fake news.

3.3 Model Selection and Cross-Validation

We evaluated six machine learning algorithms to establish a robust comparative framework:

- **Logistic Regression (LR):** An efficient linear baseline for high-dimensional sparse data.
- **Multinomial Naive Bayes (MNB):** A standard probabilistic model for text classification.
- **Linear Support Vector Machines (SVM):** A geometric model effective at finding optimal hyperplanes in high-dimensional spaces.
- **Random Forest (RF):** A bagging ensemble that captures non-linearities and resists outliers.
- **AdaBoost:** A boosting technique that iteratively corrects weak learners.
- **LightGBM:** A gradient boosting framework optimized for speed and large datasets.

We evaluated all models using a uniform Stratified 5-Fold Cross-Validation strategy. This approach maintains the dataset’s natural target equilibrium (51.5% real, 48.5% fake) across all splits, ensuring stable metrics, fair comparisons, and eliminating the need for algorithmic class balancing interventions. For ensemble models, this 5-fold architecture also handled the randomized hyperparameter search. To strictly prevent data leakage, we encapsulated all

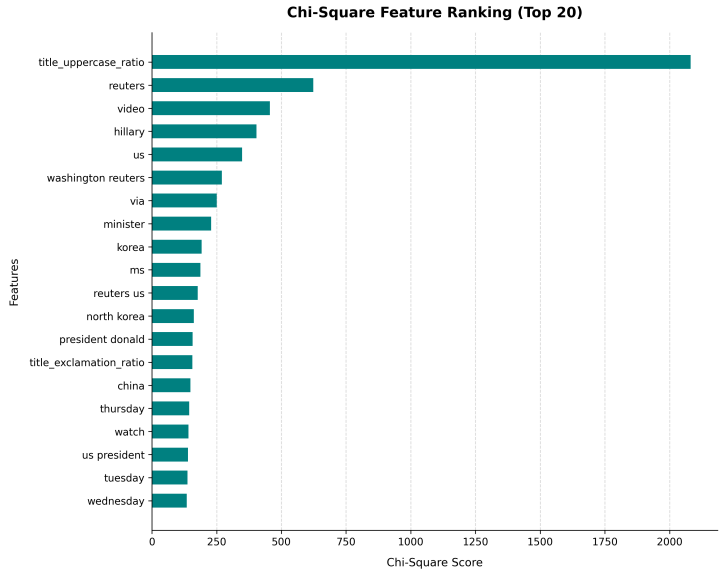

Figure 3. Top 20 feature importance scores according to Chi-Square score

vectorization and scaling steps within Scikit-Learn `Pipeline` objects. This architecture ensures that data transformations are fitted exclusively on the training folds prior to transforming the isolated validation folds. Table 2 and Figure 4 summarize the cross-validated performance of all candidate models.

Table 2. Comparison of candidate models using a uniform evaluation framework.

Model	HP Tuning	CV Strategy	Val. Accuracy	Test Accuracy
Logistic Regression	None	5-fold Stratified	0.958	0.959
Multinomial NB	None	5-fold Stratified	0.859	0.857
Linear SVM	None	5-fold Stratified	0.969	0.968
Random Forest	Random Search	5-fold Stratified	0.949	0.949
AdaBoost	Random Search	5-fold Stratified	0.952	0.952
LightGBM	Random Search	5-fold Stratified	**0.978**	**0.975**

3.4 Hyperparameter Optimization

We applied a randomized search strategy to optimize the ensemble models. The optimization framework evaluated five random parameter combinations per algorithm. We utilized accuracy to select the best configurations. The Random Forest search space included the number of estimators (50, 100, 150) and the maximum depth (10, 20, 30). The randomized search identified 150 estimators and a maximum depth of 30 as the optimal configuration. The AdaBoost tuning process evaluated the number of estimators (50, 100, 150) and the learning rate (0.1, 1.0). The optimization selected 150 estimators and a 1.0 learning rate. The LightGBM hyperparameter space spanned the number of estimators (50, 100, 150), the learning rate (0.05, 0.1), and the number of leaves (15, 31). The framework chose 100 estimators, a 0.1 learning rate, and 31 leaves. Figure 5 illustrates the validation accuracy trends across the sampled configurations.

3.5 Statistical Tests

We applied McNemar's test to compare model performances. This non-parametric test evaluates the prediction disagreement between two distinct classifiers. It analyzes the paired nominal

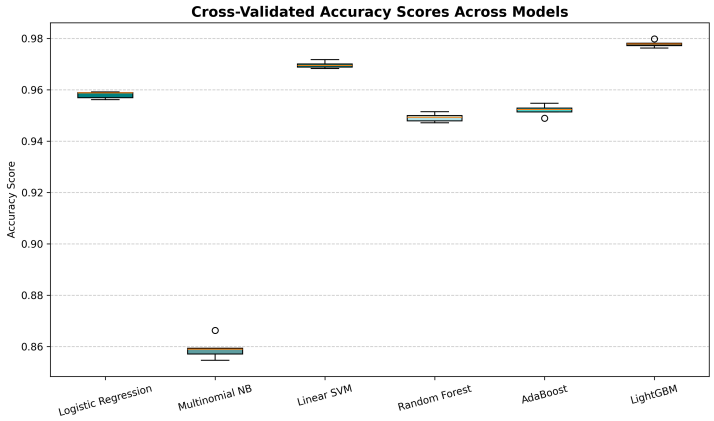

Figure 4. Box plot of cross-validated accuracy scores across candidate models using their respective stratified fold configurations.

data generated on the final test set, following the final model training on the full training set. We compared the highest-performing ensemble model (LightGBM) against the strongest linear baseline (Linear SVM). The contingency table captured the discordant predictions between the two architectures. The statistical evaluation yielded a Chi-squared test statistic (χ^2) of 20.32. The procedure returned a p-value of 0.00001. We established a significance threshold (α) of 0.05. The calculated p-value falls below this critical threshold. We reject the null hypothesis of equal predictive accuracy. This outcome confirms the statistical superiority of the LightGBM model over the baseline approach.

Table 3. McNemar's test results comparing the LightGBM and Linear SVM predictions on the test set.

Model Comparison	**Test Statistic (χ^2)**	**p-value**	**Significant ($\alpha = 0.05$)**
LightGBM vs. Linear SVM	20.32	0.00001	Yes

3.6 Final Model Training

Using the optimal hyperparameters identified via cross-validation, the highest-performing model, LightGBM, was trained on the complete 80% development set to maximize exposure to textual patterns. To guarantee an unbiased evaluation of real-world generalization, the model was then assessed on the strictly isolated 20% test set. Final performance was quantified using accuracy, precision, recall, F1-score, and AUC-ROC.

4. Results and Discussion

4.1 Performance Evaluation

We evaluated the LightGBM model on the 20% hold-out test set. The confusion matrix in Figure 6 shows that the model produces fewer False Negatives (FN) than False Positives (FP). In this context, FNs incorrectly flag legitimate articles as fake, risking censorship and damaging publisher trust, while FPs allow fabricated news to pass as real and mislead the public. The model's lower FN rate (reflected in the high recall of 0.982) indicates a conservative decision boundary that prioritizes protecting potentially legitimate content over aggressively intercepting

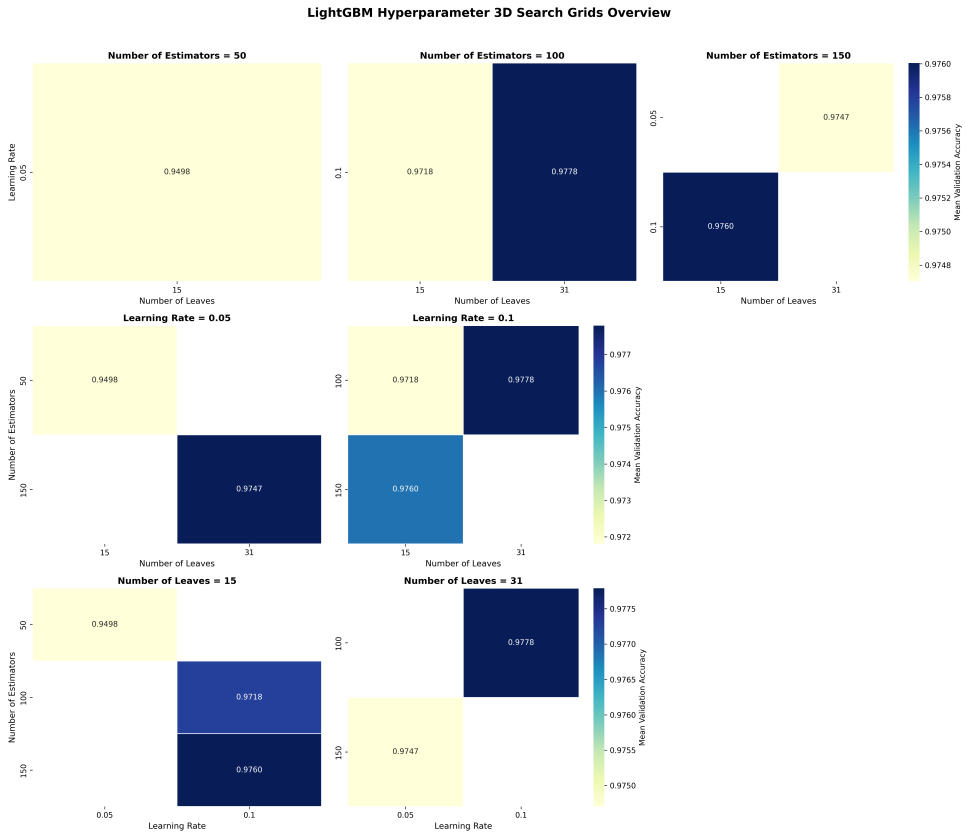

Figure 5. LightGBM Hyperparameter 3D Search Grids Overview.

all fake news. While both errors carry severe consequences, this imbalance ensures that genuine information is rarely suppressed, though it requires the moderation system to accept a slightly higher rate of occasional fake news slipping through (over-acceptance).

Table 4. Final test set performance metrics for the LightGBM model.

Metric	Score
Accuracy	0.975
Precision	0.971
Recall	0.982
F1-Score	0.976
AUC-ROC	0.997

4.2 Model Explainability

Figure 8 illustrates the global feature importance of the LightGBM model computed via SHAP values. The source token `reuters` is the most dominant predictor; its presence strongly drives predictions toward the fake news class, suggesting that deceptive articles frequently spoof agency watermarks. Among stylistic features, `title_uppercase_ratio` is the second most influential variable, unexpectedly correlating with real news. High densities of `text_exclamation_ratio` and `spelling_errors_ratio` also shift predictions toward the legitimate news domain. Conversely, text volume (`text_length`) exhibits a negligible impact, clustering near zero. This indicates that the LightGBM model relies heavily on specific source attributions and structural text anomalies rather than simple document length. At the document

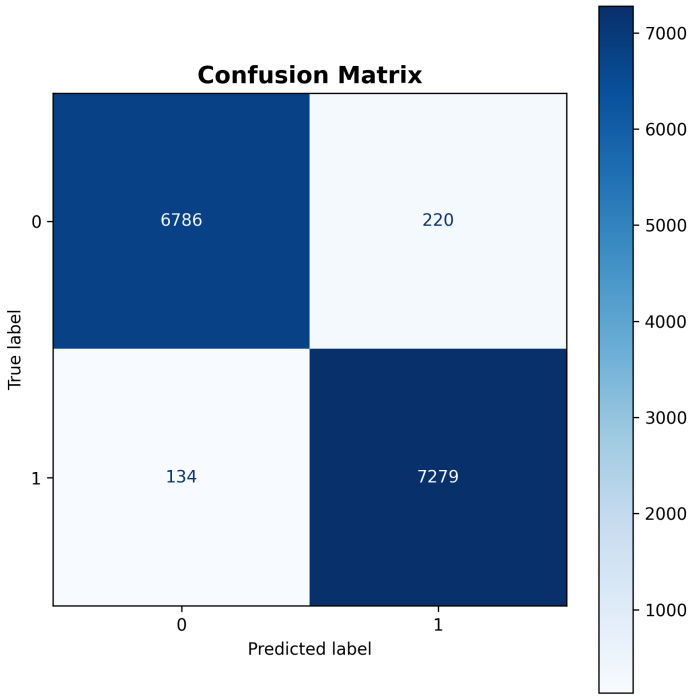

Figure 6. Confusion matrix detailing the raw prediction counts for the LightGBM model on the test set.

level, Figure 9 provides a local explanation for a single test sample (Index 2) via a SHAP waterfall plot using the LightGBM model. Starting from a base value of $E[f(X)] = 0.014$, the model reaches a final prediction of $f(x) = 6.62$, confidently classifying the article as real news. The dominant positive contribution comes from `title_uppercase_ratio` (+4.06), followed by the absence of the fake-news-associated token `reuters` (+1.14) and the presence of temporal markers like `october` (+0.57). Conversely, the token `breitbart` exerts the strongest negative push (–0.82), partially counteracting the legitimate news signal. This confirms the model operates logically on individual documents, combining structural and lexical evidence coherently. This reliance on source tokens and document length explains the observed FP and FN rates, leaving the model vulnerable to adversarial texts mimicking these structural signals.

4.3 Error Analysis

An analysis of misclassified samples reveals two primary failure patterns explained by the SHAP outputs. The first mode (**False Positives**) occurs when fake news successfully mimics legitimate reporting by exploiting stylistic and typographic signals. Specifically, elevated levels of `title_uppercase_ratio`, `text_exclamation_ratio`, or `spelling_errors_ratio`—which the model strongly associates with real news—override the deceptive context, pulling fake articles across the decision boundary into the legitimate class. Conversely, the second mode (**False Negatives**) flags genuine pieces as fake news. These articles are heavily penalized when they contain specific source-related tokens (e.g., *reuters* or *breitbart*) that the model inherently associates with misinformation, or when they lack the exaggerated typographic markers expected by the system. Ultimately, this reliance on surface-level stylistic anomalies and source identity rather than factual verification creates a clear vulnerability to adversarial manipulation, allowing structurally optimized fake news to evade detection while penalizing unconventionally written genuine content.

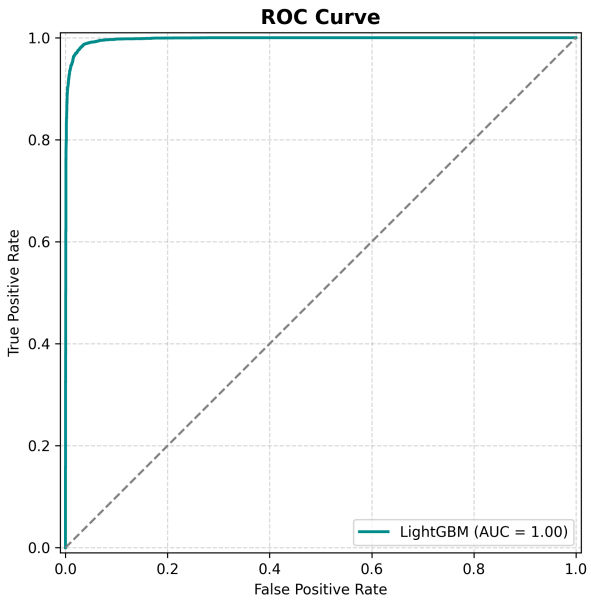

Figure 7. Receiver Operating Characteristic (ROC) curve for the final model.

5. Conclusions

This study successfully developed an automated fake news detection system, meeting all initial objectives. Our LightGBM ensemble achieved a 0.975 test accuracy, significantly outperforming all baseline and ensemble models. Statistical tests and explainability analysis confirmed both the superiority of the ensemble approach and its alignment with domain knowledge. However, the system has notable limitations. The static TF-IDF pipeline lacks deep semantic understanding, causing the model to struggle with sophisticated propaganda or deceptive texts that use trusted institutional tokens. Furthermore, it occasionally misclassifies legitimate tabloid journalism, highlighting the inherent constraints of strictly stylometric classification. Future work should address these vulnerabilities by exploring transformer-based architectures (e.g., BERT, RoBERTa) to capture deep contextual embeddings. Additionally, developing a multimodal approach that integrates image analysis and metadata verification will be crucial to enhancing classification robustness beyond the baseline established in this study.

References

[1] Allcott, H., & Gentzkow, M. (2017). Social media and fake news in the 2016 election. *Journal of Economic Perspectives*, Vol(31), pp. 211–236. https://www.aeaweb.org/articles?id=10.1257/jep.31.2.211

[2] Guo, Z., Schlichtkrull, M., & Vlachos, A. (2022). A survey on automated fact-checking. *Journal of Artificial Intelligence Research*, Vol(74), pp. 123–169. https://arxiv.org/abs/2108.11896

[3] WelFake Dataset on Kaggle: https://www.kaggle.com/datasets/saurabhshahane/fake-news-classification

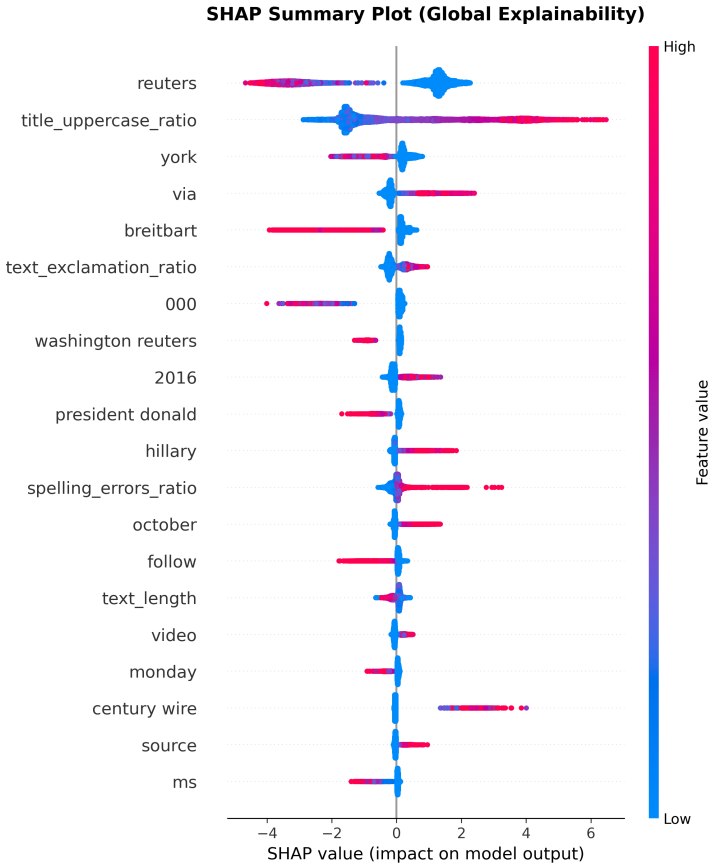

Figure 8. Global explainability: SHAP summary plot extracted from the LightGBM model.

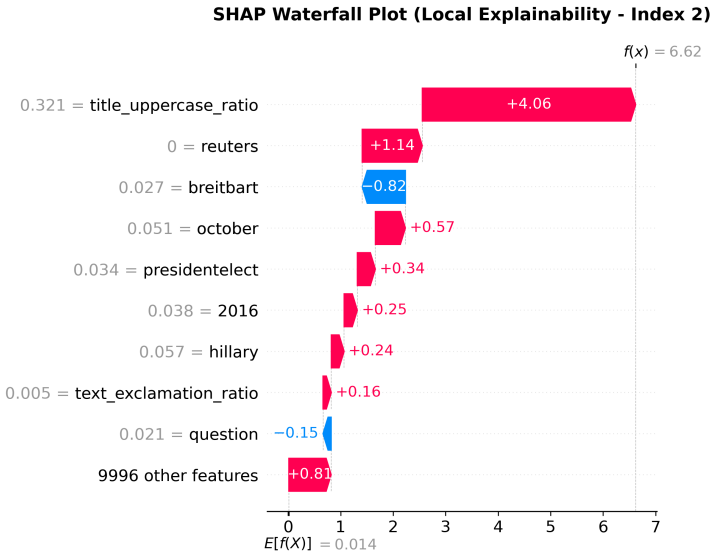

Figure 9. Local explainability: SHAP waterfall plot showcasing feature contributions for a specific article prediction from LightGBM model.